P1935R0
A C++ Approach to Physical Units

Published Proposal, 2019-10-14

This version:
https://mpusz.github.io/wg21-papers/papers/1935_a_cpp_approach_to_physical_units.html

Author:
Mateusz Pusz (Epam Systems) mateusz.pusz@gmail.com

Audience:
SG6, LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/mpusz/wg21_papers/blob/master/src/1935_a_cpp_approach_to_physical_units.bs

---

Abstract

This document starts the discussion about the Physical Units support for the C++ Standard Library. The reader will find here the rationale for such a library. After that comes the review and comparison of current solutions on the market followed by the analysis of the problems related to their usage and user experience. The rest of the document describes solutions and techniques that can be used to mitigate those issues. All of them were implemented and tested by the author in the mp-units library.

{Please note that the chapter § 4 Prior Work is still incomplete and will be filled in D1935R1 that should be available as a draft on the LEWG Wiki before Belfast meeting }

Table of Contents

§ 1. Introduction

§ 1.1. Overview

Human history knows many expensive failures and accidents caused by mistakes in calculations involving different physical units. The most famous and probably the most expensive example in the software engineering domain is the Mars Climate Orbiter that in 1999 failed to enter Mars orbit and crashed while entering its atmosphere [MARS_ORBITER]. That is not the only example here. People tend to confuse units quite often. We see similar errors occurring in various domains over the years:

- On October 12, 1492, Christopher Columbus unintentionally discovered America because during his travel preparations he mixed Arabic mile with a Roman mile which led to the wrong estimation of the equator and his expected travel distance [COLUMBUS]
- Air Canada Flight 143 ran out of fuel on July 23, 1983, at an altitude of 41 000 feet (12 000 metres), midway through the flight because the fuel had been calculated in pounds instead of kilograms by the ground crew [GIMLI_GLIDER]
- On April 15, 1999, Korean Air Cargo Flight 6316 crashed due to the miscommunication between pilots about desired flight altitude [FLIGHT_6316]
- In February 2001 Zoo crew built an enclosure for Clarence the Tortoise with a weight of 250 pounds instead of 250 kilograms [CLARENCE]

- In December 2003, one of the roller coaster's cars at Tokyo Disneyland's Space Mountain attraction suddenly derailed due to a broken axle caused by the confusion after upgrading the specification from imperial to metric units [DISNEY]
- An American company sold a shipment of wild rice to a Japanese customer, quoting a price of 39 cents per pound, but the customer thought the quote was for 39 cents per kilogram [WILD_RICE]
- A whole set of medication dose errors...

§ 1.2. Lack of strong types

It turns out that in the C++ software most of our calculations in the physical units domain are handled with fundamental types like `double`. Code like below is a typical example here:

```
double GlidePolar::MacCreadyAltitude(double emcready,
                                     double Distance,
                                     const double Bearing,
                                     const double WindSpeed,
                                     const double WindBearing,
                                     double *BestCruiseTrack,
                                     double *VMacCready,
                                     const bool isFinalGlide,
                                     double *TimeToGo,
                                     const double AltitudeAboveTarget,
                                     const double cruise_efficiency,
                                     const double TaskAltDiff);
```

Even though this example comes from an Open Source project, expensive revenue-generating production source code often does not differ too much. We lack strong typedefs feature in the core language, and without it, we are often too lazy to handcraft a new class type for each use case.

§ 1.3. The proliferation of magic numbers

There are a lot of constants and conversion factors involved in the dimensional analysis. Source code responsible for such computations is often trashed with magic numbers

```
// Air Density(kg/m3) from relative humidity(%),
// temperature(°C) and absolute pressure(Pa)
double AirDensity(double hr, double temp, double abs_press)
{
  return (1/(287.06*(temp+273.15))) *
         (abs_press - 230.617 * hr * exp((17.5043*temp)/(241.2+temp)));
}
```

§ 2. Motivation and Scope

§ 2.1. Motivation

There is a huge demand for high-quality physical units library in the industry and scientific environments. The code that we write for fun and living should be correct, safe, and easy to write. Although there are multiple such libraries available on the market, none of them is a widely accepted production standard. We could just provide a yet another 3rd party library covering this topic, but it is probably not the best idea.

First of all, software that could benefit from such a library is not a niche in the market. If it was the case, probably its needs could be fulfilled with a 3rd party highly-specialized and narrow-use library. On the contrary, a broad range of production projects deals with units conversions and dimensional analysis. Right now, having no other reasonable and easy to access alternatives results in the proliferation of plain `double` type usage to express physical quantities. Space, aviation, automotive, embedded, scientific, computer science, and many other domains could benefit from strong types and conversions provided by such a library.

Secondly, yet another library will not solve the issue for many customers. Many corporations are not allowed to use 3rd party libraries in the production code. Also, an important point here is the cooperation of different products from multiple vendors that use physical quantities as vocabulary types in their interfaces. From the author's experience gathered while working with numerous corporations all over the world, there is a considerable difference between the adoption of a mature 3rd party library and the usage of features released as a part of the C++ Standard Library. If it were not the case all products would use Boost.Units already. A motivating example here can be `std::chrono` released as a part of C++11. Right now, no one asks questions on how to represent timestamps and how to handle their conversions in the code. `std::chrono` is the ultimate answer. So let us try to get `std::units` in the C++ Standard Library too.

§ 2.2. The Goal

The aim of this paper is to standardize a physical units library that enables operations on various dimensions and units:

```
// simple numeric operations
static_assert(10km / 2 == 5km);

// unit conversions
static_assert(1h == 3600s);
static_assert(1km + 1m == 1001m);

// dimension conversions
static_assert(1km / 1s == 1000mps);
static_assert(2kmph * 2h == 4km);
static_assert(2km / 2kmph == 1h);

static_assert(1000 / 1s == 1kHz);

static_assert(10km / 5km == 2);
```

We intent to provide users with cleaner interfaces by using strong types and concepts in the interfaces rather than fundamental types with meaning described in comments or documentation:

```
constexpr std::units::Velocity auto avg_speed(std::units::Length auto d, std::units::Time a
{
  return d / t;
}
```

We further aim to provide unit conversion facilities and constants for users to rely on, instead of magic numbers:

```
using namespace std::units_literals;

const std::units::Velocity auto speed = avg_speed(220.km, 2.h);
std::cout << "Average speed: "
          << std::units::quantity_cast<std::units::kilometre_per_hour>(speed) << '\n';
```

§ 2.3. Scope

Although there is a public demand for a generic units library that could handle any units and dimensions, the author suggests scoping the Committee efforts only on the physical and possibly computer science (i.e. `bit`, `byte`, `bitrate`) units first. The library should be designed with easy extensibility in mind so anyone needing a new base or derived dimensions (i.e. `coffee/milk/water/sugar` system) could achieve this with a few lines of the C++ code (not preprocessor macros).

After releasing a first, restricted version of the library and observing how it is used we can consider standardizing additional dimensions, units, and constants in the following C++ releases.

§ 3. Terms and definitions

§ 3.1. ISO 80000-1:2009(E) definitions

ISO 80000-1:2009(E) Quantities and units - Part 1: General [ISO_80000-1] defines among others the following terms:

quantity

- Property of a phenomenon, body, or substance, where the property has a magnitude that can be expressed by means of a number and a reference.
- A reference can be a measurement unit, a measurement procedure, a reference material, or a combination of such.
- A quantity as defined here is a scalar. However, a vector or a tensor, the components of which are quantities, is also considered to be a quantity.
- The concept ’quantity’ may be generically divided into, e.g. ‘physical quantity’, ‘chemical quantity’, and ‘biological quantity’, or ‘base quantity’ and ‘derived quantity’.
- Examples of quantities are: mass, length, density, magnetic field strength, etc.

kind of quantity, kind

- Aspect common to mutually comparable quantities.
- The division of the concept ‘quantity’ into several kinds is to some extent arbitrary

    - i.e. the quantities diameter, circumference, and wavelength are generally considered to be quantities of the same kind, namely, of the kind of quantity called length.)
- Quantities of the same kind within a given system of quantities have the same quantity dimension. However, quantities of the same dimension are not necessarily of the same kind.
    - For example, the absorbed dose and the dose equivalent have the same dimension. However, the former measures the absolute amount of radiation one receives wherase the latter is a weighted measurement taking into account the kind of radiation on was exposed to.

system of quantities, system

- Set of quantities together with a set of non-contradictory equations relating those quantities.
- Examples of systems of quantities are: the International System of Quantities, the Imperial System, etc.

base quantity

- Quantity in a conventionally chosen subset of a given system of quantities, where no quantity in the subset can be expressed in terms of the other quantities within that subset.
- Base quantities are referred to as being mutually independent since a base quantity cannot be expressed as a product of powers of the other base quantities.

derived quantity

- Quantity, in a system of quantities, defined in terms of the base quantities of that system.

International System of Quantities (ISQ)

- System of quantities based on the seven base quantities: length, mass, time, electric current, thermodynamic temperature, amount of substance, and luminous intensity.
- The International System of Units (SI) is based on the ISQ.

dimension of a quantity, quantity dimension, dimension

- Expression of the dependence of a quantity on the base quantities of a system of quantities as a product of powers of factors corresponding to the base quantities, omitting any numerical factors.
- A power of a factor is the factor raised to an exponent. Each factor is the dimension of a base quantity.
- In deriving the dimension of a quantity, no account is taken of its scalar, vector, or tensor character.
- In a given system of quantities:
    - quantities of the same kind have the same quantity dimension,
    - quantities of different quantity dimensions are always of different kinds,
    - quantities having the same quantity dimension are not necessarily of the same kind.

quantity of dimension one, dimensionless quantity

- Quantity for which all the exponents of the factors corresponding to the base quantities in its quantity dimension are zero.
- The term “dimensionless quantity” is commonly used and is kept here for historical reasons. It stems from the fact that all exponents are zero in the symbolic representation of the dimension for such quantities. The term “quantity of

dimension one" reflects the convention in which the symbolic representation of the dimension for such quantities is the symbol 1. This dimension is not a number, but the neutral element for multiplication of dimensions.

- The measurement units and values of quantities of dimension one are numbers, but such quantities convey more information than a number.
- Some quantities of dimension one are defined as the ratios of two quantities of the same kind. The coherent derived unit is the number one, symbol 1.
- Numbers of entities are quantities of dimension one.

unit of measurement, measurement unit, unit

- Real scalar quantity, defined and adopted by convention, with which any other quantity of the same kind can be compared to express the ratio of the second quantity to the first one as a number.
- Measurement units are designated by conventionally assigned names and symbols.
- Measurement units of quantities of the same quantity dimension may be designated by the same name and symbol even when the quantities are not of the same kind. For example, joule per kelvin and J/K are respectively the name and symbol of both a measurement unit of heat capacity and a measurement unit of entropy, which are generally not considered to be quantities of the same kind. However, in some cases special measurement unit names are restricted to be used with quantities of specific kind only. For example, the measurement unit 'second to the power minus one' (1/s) is called hertz (Hz) when used for frequencies and becquerel (Bq) when used for activities of radionuclides. As another example, the joule (J) is used as a unit of energy, but never as a unit of moment of force, i.e. the newton metre (N · m).
- Measurement units of quantities of dimension one are numbers. In some cases, these measurement units are given special names, e.g. radian, steradian, and decibel, or are expressed by quotients such as millimole per mole equal to 10^{-3} and microgram per kilogram equal to 10^{-9}.

base unit

- Measurement unit that is adopted by convention for a base quantity.
- In each coherent system of units, there is only one base unit for each base quantity.
- A base unit may also serve for a derived quantity of the same quantity dimension.
- For example, the ISQ has the base units of: metre, kilogram, second, Ampere, Kelvin, mole, and candela.

derived unit

- Measurement unit for a derived quantity.
- For example, in the ISQ Newton, Pascal, and katal are derived units.

coherent derived unit

- Derived unit that, for a given system of quantities and for a chosen set of base units, is a product of powers of base units with no other proportionality factor than one.
- A power of a base unit is the base unit raised to an exponent.
- Coherence can be determined only with respect to a particular system of quantities and a given set of base units. That is, if the metre and the second are base units, the metre per second is the coherent derived unit of velocity.

system of units

- Set of base units and derived units, together with their multiples and submultiples, defined in accordance with given rules, for a given system of quantities.

off-system measurement unit, off-system unit

- Measurement unit that does not belong to a given system of units. For example, the electronvolt (≈ 1,602 18 × 10–19 J) is an off-system measurement unit of energy with respect to the SI or day, hour, minute are off-system measurement units of time with respect to the SI.

International System of Units (SI)

- System of units, based on the International System of Quantities, their names and symbols, including a series of prefixes and their names and symbols, together with rules for their use, adopted by the General Conference on Weights and Measures (CGPM)

multiple of a unit

- Measurement unit obtained by multiplying a given measurement unit by an integer greater than one.
- SI prefixes refer strictly to powers of 10, and should not be used for powers of 2. That is, 1 kbit should not be used to represent 1024 bits (2^{10} bits), which is a kibibit (1 Kibit).

submultiple of a unit

- Measurement unit obtained by dividing a given measurement unit by an integer greater than one.

quantity value, value of a quantity, value

- Number and reference together expressing magnitude of a quantity.
- A quantity value can be presented in more than one way.

§ 4. Prior Work

There are multiple dimensional analysis libraries available on the market today. Some of them are more successful than others, but none of them is a widely accepted standard in the C++ codebase (both for Open Source as well as production code). The next sections of this chapter will describe the most interesting parts of selected libraries. The last section provides an extensive comparison of their main features.

{This chapter is incomplete and will be filled in D1935R1 that should be available as a draft on the LEWG Wiki before Belfast meeting}

§ 4.1. Boost.Units

Boost.Units [BOOST.UNITS] is probably the most widely adopted library in this domain. It was first released in Boost 1.36.0 that was released in 2008.

{TBD}

§ 4.2. cppnow17-units

Steven Watanabe, the coauthor of the previous library, started the work on the modernized version of the library based on the results of LiaW on C++Now 2017 [CPPNOW17-UNITS]

{TBD}

§ 4.3. PhysUnits-CT-Cpp11

[PHYSUNITS-CT-CPP11]

{TBD}

§ 4.4. Nic Holthaus units

The next library created by Nic Holthaus [NIC_UNITS] provides a dimension as a hardcoded sequence of `std::ratio`s in a `base_unit` class template.

```
namespace units {

template<class Meter = detail::meter_ratio<0>,
         class Kilogram = std::ratio<0>,
         class Second = std::ratio<0>,
         class Radian = std::ratio<0>,
         class Ampere = std::ratio<0>,
         class Kelvin = std::ratio<0>,
         class Mole = std::ratio<0>,
         class Candela = std::ratio<0>,
         class Byte = std::ratio<0>>
struct base_unit;

}
```

Unit is expressed as instantiations of the `unit` class template.

```
namespace units {

template<class Conversion, class BaseUnit, class PiExponent = std::ratio<0>, class Translat

}
```

Important to notice here are:

- `PiExponent` - an exponent representing factors of PI required by the conversion (i.e. `std::ratio<-1>` for a radians to degrees conversion)
- `Translation` - a ratio representing a datum translation required for the conversion (i.e. `std::ratio<32>` for a Fahrenheit to Celsius conversion)

```
namespace units {

template<class Units, typename T = UNIT_LIB_DEFAULT_TYPE,
         template<typename> class NonLinearScale = linear_scale>
class unit_t : public NonLinearScale<T> { ... };

}
```

Interesting to notice here is that beside typical SI dimensions, there are also `Radian` and `Byte`.

This library also presents a different approach than previous cases. There are no dimensions or quantity types. Every unit is an instantiation of the `unit` class template with ratio and a specific `base_unit` responsible for unit "category". Each "dimension" of the unit is defined in its namespace. To form a quantity, there is additional

```
namespace units {

namespace category {

typedef base_unit<std::ratio<2>, std::ratio<1>, std::ratio<-3>, std::ratio<0>, std::ratio<-

}

namespace voltage {

typedef unit<std::ratio<1>, units::category::voltage_unit> volts;
typedef volts volt;
typedef unit_t<volt> volt_t;

}

}
```

To form a value

```
#include <units.h>

using namespace units::literals;

units::voltage::volt_t v = 230_V;
```

§ 4.5. Bryan St. Amour units

[BRYAN_UNITS]

{TBD}

§ 4.6. benri

[BENRI]

{TBD}

§ 4.7. Michael Ford units

[MIKEFORD3_UNITS]

{TBD}

§ 4.8. `std::chrono::duration`

{TBD}

§ 4.9. Comparison

{TBD}

Feature	mp-units	Boost.Units	cppnow17-units	PhysUnits-CT-Cpp11>
SI	yes	yes	yes	yes
Customary system	yes	yes		some
Other systems	???	yes		some
C++ version	C++20	C++98 + `constexpr`		C++11
Base dimension id	string	integer		
Dimension	type (`length`)	type (`length_dimension`)		
Dimension representation	type list	type list		
Fractional exponents	yes	yes		
Type traits for dimensions	no	yes		
Unit	type (`metre`)	type + constant (`si::length` + `si::meter`)		
UDLs	yes	no		
Composable UDLs	no (`kilometre`)	no		
Predefined scaled unit types	some	no		
Scaled units	type + UDL (`kilometre` + `km`)	user's type + multiply with constant (`make_scaled_unit<>` + `si::kilo * si::meter`)		
Meter vs metre	metre	both		
Singular vs plural	singular (`metre`)	both (`meter` + `meters`)		
Quantity	type (`quantity<metre> q(2);`)	type (`quantity<si::length> q(2 * si::meter);`)		

Literal instance	UDL (`123m`)	Number * static constant (`123 * si::meters`)		
Variable instance	constructor (`quantity<metre>(v)`)	Variable * static constant (`d * si::meters`)		
Any representation	yes	yes		no (macro to set the default type
Quantity template arguments type deduction	yes	yes		
System support	no	yes		
C++ Concepts	yes	no	no	no
Types downcasting	yes	no	no	no
Implicit unit conversions	same dimension non-truncating only	no		
Explicit unit conversions	`quantity_cast`	`quantity_cast`		
Temperature support	Kelvins only + conversion functions	Kelvins only + dedicated systems		
String output	TBD	yes		
String input	no	no		
Macros in the user interface	no	yes		
Non-linear scale support	no	no		
`<cmath>` support	no	yes		no
`<chrono>` support	no	no		no
Affine types	no	yes (`TBD`, `TBD`)		no
Prefix representation				
Physical/Mathematical constants	no	yes		limited
Dimensionless quantity	no			
Arbitrary conversions		yes		yes
User defined dimensions	yes	yes		no
User defined units	yes			
User defined prefixes	yes	yes		xes

Feature	**mp-units**	**nholthaus**	**bstamour**	**benri**	**mfor units**
SI	yes	yes	yes	yes	
Customary system	yes	yes		yes	
Other systems	???	yes (`bytes`, `radians`)		yes	
C++ version	C++20	C++14		C++14	
Base dimension id	string	index on template parameter list		string	
Dimension	type (`length`)	none		alias to type list (`length_t`)	

Dimension representation	type list	Class template arguments		type list
Fractional exponents	yes	yes		yes
Type traits for dimensions	no	yes		some
Unit	type (`metre`)	type (`length::meter_t`)		alias to type (`metre_t`)
UDLs	yes	yes		yes
Composable UDLs	no (`kilometre`)			yes (`kilo * metre`)
Predefined scaled unit types	some	all		
Scaled units	type + UDL (`kilometre` + `km`)	type + UDL (`length::kilometer_t` + `_km`)		
Meter vs metre	metre	meter		metre
Singular vs plural	singular (`metre`)	both (`length::meter_t` + `length::meters_t`)		singular (`metre`)
Quantity	type (`quantity<metre> q(2);`)	value of unit (`length::meter_t d(220);`)		type (`quantity<metre> q(2);`)
Literal instance	UDL (`123m`)	UDL (`123_m`)		UDL (`1_metre`)
Variable instance	constructor (`quantity<metre>(v)`)	constructor (`length::meter_t(v)`)		constructor (`quantity<metre>(v)`)
Any representation	yes	no (macro to set the default type)		yes (macro default of `double`)
Quantity template arguments type deduction	yes	no		
System support	no	no		no
C++ Concepts	yes	no	no	no
Types downcasting	yes	no	no	no
Implicit unit conversions	same dimension non-truncating only			no
Explicit unit conversions	`quantity_cast`			`simple_cast` (`constexpr`)/`unit_cast`
Temperature support	Kelvins only + conversion functions			yes
String output	TBD	yes		no
String input	no	no		no
Macros in the user interface	no	yes		yes
Non-linear scale support	no	yes		
`<cmath>` support	no	yes		yes
`<chrono>` support	no	yes		yes

Affine types	no	no	yes (`quantity`, `quantity_point`)
Prefix representation			type list
Physical/Mathematical constants	no	limited	all
Dimensionless quantity	no		yes
Arbitrary conversions		no	yes
User defined dimensions	yes	no	yes
User defined units	yes		yes
User defined prefixes	yes	yes	yes

§ 5. Fundamental concerns with current solutions

Feedback from the users gathered so far signals the following significant complaints regarding the libraries described in § 4 Prior Work:

1. Bad user experience caused by hard to understand and analyze compile-time errors and poor debugging experience (addressed by § 6 Improving user experience).
2. Unnecessary intermediate quantity value conversions to base units resulting in a runtime overhead and loss of precision (addressed by § 7 Limiting intermediate quantity value conversions).
3. Poor support for really large or small unit ratios (i.e. `eV`) (addressed by § 8 std::ratio on steroids).
4. Impossibility or hard extensibility of the library with new base quantities (addressed by § 9 Extensibility).
5. Too high entry bar (e.g. Boost.Units is claimed to require expertise in both C++ and dimensional analysis) (addressed by § 10 Easy to use and hard to abuse).
6. Safety and security connected problems with the usage of an external 3rd party library for production purposes (addresed by § 2.1 Motivation).

§ 6. Improving user experience

§ 6.1. Type aliasing issues

Type aliases benefit developers but not end-users. As a result users end up with colossal error messages.

Taking Boost.Units as an example, the code developer works with the following syntax:

```
namespace bu = boost::units;

constexpr bu::quantity<bu::si::velocity> avg_speed(bu::quantity<bu::si::length> d,
                                                   bu::quantity<bu::si::time> t)
{ return d * t; }
```

Above calculation contains a simple error as a velocity derived quantity cannot be created from multiplication of length and time base quantities. If such an error happens in the source code, user will need to analyze the following error for gcc-8:

```
error: could not convert ‘boost::units::operator*(const boost::units::quantity<Unit1, X>&,
const boost::units::quantity<Unit2, Y>&) [with Unit1 = boost::units::unit<boost::units::lis
<boost::units::length_base_dimension, boost::units::static_rational<1> >, boost::units::dim
boost::units::homogeneous_system<boost::units::list<boost::units::si::meter_base_unit,
boost::units::list<boost::units::scaled_base_unit<boost::units::cgs::gram_base_unit, boost:
boost::units::static_rational<3> > >, boost::units::list<boost::units::si::second_base_unit
boost::units::list<boost::units::si::ampere_base_unit, boost::units::list<boost::units::si:
boost::units::list<boost::units::si::mole_base_unit, boost::units::list<boost::units::si::c
boost::units::list<boost::units::angle::radian_base_unit, boost::units::list<boost::units::
boost::units::dimensionless_type> > > > > > > > > >; Unit2 = boost::units::unit<boost::un
<boost::units::time_base_dimension, boost::units::static_rational<1> >, boost::units::dimen
boost::units::homogeneous_system<boost::units::list<boost::units::si::meter_base_unit,
boost::units::list<boost::units::scaled_base_unit<boost::units::cgs::gram_base_unit, boost:
boost::units::static_rational<3> > >, boost::units::list<boost::units::si::second_base_unit
<boost::units::si::ampere_base_unit, boost::units::list<boost::units::si::kelvin_base_unit,
<boost::units::si::mole_base_unit, boost::units::list<boost::units::si::candela_base_unit,
<boost::units::angle::radian_base_unit, boost::units::list<boost::units::angle::steradian_b
boost::units::dimensionless_type> > > > > > > > > >; X = double; Y = double; typename
boost::units::multiply_typeof_helper<boost::units::quantity<Unit1, X>, boost::units::quanti
boost::units::quantity<boost::units::unit<boost::units::list<boost::units::dim<boost::units
boost::units::static_rational<1> >, boost::units::list<boost::units::dim<boost::units::time
boost::units::static_rational<1> >, boost::units::dimensionless_type> >, boost::units::homo
<boost::units::list<boost::units::si::meter_base_unit, boost::units::list<boost::units::sca
<boost::units::cgs::gram_base_unit, boost::units::scale<10, boost::units::static_rational<3
boost::units::list<boost::units::si::second_base_unit, boost::units::list<boost::units::si:
boost::units::list<boost::units::si::kelvin_base_unit, boost::units::list<boost::units::si:
boost::units::list<boost::units::si::candela_base_unit, boost::units::list<boost::units::an
boost::units::list<boost::units::angle::steradian_base_unit, boost::units::dimensionless_ty
void>, double>](t)’ from ‘quantity<unit<list<[...],list<dim<[...],static_rational<1>>,[...]
to ‘quantity<unit<list<[...],list<dim<[...],static_rational<-1>>,[...]>>,[...],[...]>,[...]
     return d * t;
            ~~^~~
```

An important point to notice here is that above text is just the very first line of the compilation error log. Error log for the same problem generated by clang-7 looks as follows:

```
error: no viable conversion from returned value of type 'quantity<unit<list<[...], list<dim
static_rational<1, [...]>>, [...]>>, [...]>, [...]>' to function return type 'quantity<unit
static_rational<-1, [...]>>, [...]>>, [...]>, [...]>'
    return d * t;
           ^~~~~
```

Despite being shorter, this message does not really help much in finding the actual fault too.

Omnipresent type aliasing does not affect only compilation errors observed by the end-user but also debugging. Here is how a breakpoint for the above function looks like in the gdb debugger:

```
Breakpoint 1, avg_speed<boost::units::heterogeneous_system<boost::units::heterogeneous_syst
<boost::units::list<boost::units::heterogeneous_system_dim<boost::units::si::meter_base_uni
boost::units::dimensionless_type>, boost::units::list<boost::units::dim<boost::units::lengt
boost::units::static_rational<1> >, boost::units::dimensionless_type>, boost::units::list<b
<boost::units::scale<10, boost::units::static_rational<3> > >, boost::units::dimensionless_
boost::units::heterogeneous_system<boost::units::heterogeneous_system_impl<boost::units::li
<boost::units::heterogeneous_system_dim<boost::units::scaled_base_unit<boost::units::si::se
boost::units::scale<60, boost::units::static_rational<2> > >, boost::units::static_rational
boost::units::dimensionless_type>, boost::units::list<boost::units::dim<boost::units::time_
boost::units::static_rational<1> >, boost::units::dimensionless_type>, boost::units::dimens
velocity_2.cpp:39
39        return d / t;
```

§ 6.2. Downcasting facility

To provide much shorter error messages the author of the paper with the help of Richard Smith, implemented a downcast facility in [MP-UNITS]. It allowed converting the following error log from:

```
[with T = units::quantity<units::unit<units::dimension<units::exp<units::base_dim_length, 1
units::exp<units::base_dim_time, -1> > >, std::ratio<1> >, double>]
```

into:

```
[with T = units::quantity<units::metre_per_second, double>]
```

As a result the type dumped in the error log is exactly the same entity that the developer used to implement the erroneous source code.

The above is possible thanks to the fact that the downcasting facility provides a type substitution mechanism. It connects a specific primary class template instantiation with a strong type assigned by the user. A simplified mental model of the facility may be represented as:

```
struct velocity : derived_dimension<exp<base_dim_length, 1>, exp<base_dim_time, -1>>;
struct metre_per_second : derived_unit<velocity, std::ratio<1>>;
```

In the above example, `velocity` and `metre_per_second` are the downcasting targets (child classes), and specific `derived_dimension` and `derived_unit` class template instantiations are downcasting sources (base classes). The downcasting facility provides one to one type substitution mechanism for those types. This means that only one child class can be created for a specific base class template instantiation.

The downcasting facility is provided through two dedicated types, a concept, and a few helper template aliases.

```
template<typename BaseType>
struct downcast_base {
  using base_type = BaseType;
  friend auto downcast_guide(downcast_base);
};
```

`units::downcast_base` is a class that implements the CRTP idiom, marks the base of the downcasting facility with a `base_type` member type, and provides a declaration of the downcasting ADL friendly (Hidden Friend) entry point member function `downcast_guide`. An important design point is that this function does not return any specific type in its declaration. This non-member function is going to be defined in a child class template `downcast_helper` and will return a target type of the downcasting operation there.

```
template<typename T>
concept Downcastable =
    requires {
      typename T::base_type;
    } &&
    std::derived_from<T, downcast_base<typename T::base_type>>;
```

`units::Downcastable` is a concept that verifies if a type implements and can be used in a downcasting facility.

```
template<typename Target, Downcastable T>
struct downcast_helper : T {
  friend auto downcast_guide(typename downcast_helper::downcast_base) { return Target(); }
};
```

`units::downcast_helper` is another CRTP class template that provides the implementation of a non-member friend function of the `downcast_base` class template, which defines the target type of a downcasting operation. It is used in the following way to define `dimension` and `unit` types in the library:

```
template<typename Child, Exponent... Es>
struct derived_dimension : downcast_helper<Child, detail::make_dimension_t<Es...>> {};
```

```
template<typename Child, Dimension D>
struct derived_unit<Child, D, R> : downcast_helper<Child, unit<D, ratio<1>>> {};
```

With such helper types, the only thing the user has to do is to register a new type for the downcasting facility by publicly deriving from one of those CRTP types and provide its new child type as the first template parameter of the CRTP type:

```
struct velocity : derived_dimension<velocity, exp<base_dim_length, 1>, exp<base_dim_time, -
struct metre_per_second : derived_unit<metre_per_second, velocity, std::ratio<1>>;
```

The above types are used to define the base and target of a downcasting operation. To perform the actual downcasting operation, a dedicated template alias is provided and used by the library's framework:

```
template<Downcastable T>
using downcast_target = decltype(detail::downcast_target_impl<T>());
```

`units::downcast_target` is used to obtain the target type of the downcasting operation registered for a given instantiation in a base type.

For example, to determine a downcasted type of a quantity multiplication, the following can be done:

```
using dim = dimension_multiply<typename U1::dimension, typename U2::dimension>;
using ratio = ratio_multiply<typename U1::ratio, typename U2::ratio>;
using common_rep = decltype(lhs.count() * rhs.count());
using ret = quantity<downcast_target<unit<dim, ratio>>, common_rep>;
```

`detail::downcast_target_impl` checks if a downcasting target is registered for the specific base class. If registered, `detail::downcast_target_impl` returns the registered type, otherwise it returns the provided base class.

```
namespace detail {

  template<typename T>
  concept has_downcast = requires {
    downcast_guide(std::declval<downcast_base<T>>());
  };

  template<typename T>
  constexpr auto downcast_target_impl()
  {
    if constexpr(has_downcast<T>)
      return decltype(downcast_guide(std::declval<downcast_base<T>>()))();
    else
      return T();
  }

}
```

§ 6.3. Template instantiation issues

C++ is known for massive error logs caused by compilation errors deep down in the stack of function template instantiations of an implementation. In the vast majority of cases, this is caused by function templates just taking a `typename T` as their parameter, not placing any constratints on the actual type. In C++17 placing such constraints is possible thanks to SFINAE and helpers like `std::enable_if` or `std::void_t`. However, these are known to be not really user-friendly.

Consider the following example:

```
template<typename Length, typename Time,
         typename = std::enable_if_t<units::traits::is_length_unit<Length>::value &&
                                     units::traits::is_time_unit<Time>::value>>
constexpr auto avg_speed(Length d, Time t)
  -> std::enable_if_t<units::traits::is_velocity_unit<decltype(d / t)>::value>, decltype(d
{
  const auto v = d / t;
  static_assert(units::traits::is_velocity_unit<decltype(v)>::value);
  return v;
}
```

Clearly this is not the most user-friendly way to write code every day. Imagine the effort involved for C++ experts and non-experts alike to write longer and more complex functions, multiline calculations, or even whole programs in this style. Obviously C++20 concepts radically simplify the boiler plate involved and are thus the way to go.

§ 6.4. Better errors with C++20 concepts

With C++20 concepts above example is simplified to:

```
template<units::Length L, units::Time T>
constexpr units::Velocity auto avg_speed(L d, T t)
{
  return d / t;
}
```

Using generic functions, it can even be implemented, without the template syntax, as:

```
constexpr units::Velocity auto avg_speed(units::Length auto d, units::Time auto t)
{
  return d / t;
}
```

Thanks to C++20 concepts we not only get much stronger interfaces with their compile-time contracts clearly expressed by concepts in the function template signature, but also much better error logs. Concept constraint validation being done early in the function instantiation process catches errors early and not deep in the instantiation stack, significantly improving the readability of the actual errors.

For example, gcc with experimental Concepts TS support generates the following message:

```
example.cpp: In instantiation of ‘constexpr units::Velocity avg_speed(D, T)
    [with D = units::quantity<units::kilometre>; T = units::quantity<units::hour>]’:
example.cpp:49:49:   required from here
example.cpp:34:14: error: placeholder constraints not satisfied
    34 |   return d * t;
       |              ^
include/units/dimensions/velocity.h:34:16: note: within ‘template<class T> concept units::V
    [with T = units::quantity<units::unit<units::dimension<units::exp<units::base_dim_lengt
                units::exp<units::base_dim_time, 1, 1> >, units::ratio<3600000, 1> >, doubl
    34 |   concept Velocity = Quantity<T> && std::same_as<typename T::dimension, velocity>;
       |           ^~~~~~~~
include/stl2/detail/concepts/core.hpp:37:15: note: within ‘template<class T, class U> conce
    [with T = units::dimension<units::exp<units::base_dim_length, 1, 1>, units::exp<units::
          U = units::velocity]’
    37 |  META_CONCEPT same_as = meta::Same<T, U> && meta::Same<U, T>;
       |               ^~~~~~~
include/meta/meta_fwd.hpp:224:18: note: within ‘template<class T, class U> concept meta::Sa
    [with T = units::dimension<units::exp<units::base_dim_length, 1, 1>, units::exp<units::
          U = units::velocity]’
   224 |     META_CONCEPT Same =
       |                  ^~~~
include/meta/meta_fwd.hpp:224:18: note: ‘meta::detail::barrier’ evaluated to false
include/meta/meta_fwd.hpp:224:18: note: within ‘template<class T, class U> concept meta::Sa
    [with T = units::velocity;
          U = units::dimension<units::exp<units::base_dim_length, 1, 1>, units::exp<units::
include/meta/meta_fwd.hpp:224:18: note: ‘meta::detail::barrier’ evaluated to false
```

While still being a little verbose, this is a big improvement to the page-long instantation lists shown above. The user gets the exact information of what was wrong with the provided type, why it did not meet the required constraints, and where the error occured. With concept suppport still being experimental, we expect error message to improve even more in the future.

§ 7. Limiting intermediate quantity value conversions

Many of the physical units libraries on the market decide to quietly convert different units to the one fixed, coherent derived unit of the dimension. For example:

```
namespace bu = boost::units;

constexpr bu::quantity<bu::si::velocity> avg_speed(bu::quantity<bu::si::length> d,
                                                   bu::quantity<bu::si::time> t)
{
  return d / t;
}
```

The code always (implicitly) converts incoming `d` length and `t` time arguments to the base units of their dimensions. So if the user intends to write the code like:

```
using kilometer_base_unit = bu::make_scaled_unit<bu::si::length,
                                                 bu::scale<10, bu::static_rational<3>>>::ty
using length_kilometer    = kilometer_base_unit::unit_type;
using time_hour           = bu::metric::hour_base_unit::unit_type;
using kilometers_per_hour = bu::divide_typeof_helper<length_kilometer, time_hour>::type;
BOOST_UNITS_STATIC_CONSTANT(hours, time_hour);

const auto v = avg_speed(bu::quantity<bu::si::length>(220 * bu::si::kilo * bu::si::meters),
                         bu::quantity<bu::si::time>(2 * hours));
const bu::quantity<velocity_kilometers_per_hour> kmph(v);
std::cout << kmph.value() << " km/h\n";
```

All the values provided as arguments are first converted to SI base units before the function executes. After the function returns, the result is converted back to the same units as provided by the user for the input arguments. These conversions can significantly slow down the execution of a function, and lead to an increased loss of precision.

For our example, three conversions have to be made. One to convert the length from `220km` to `220000m`, one to convert the time from `2h` to `7200s`, and one to convert the result back from `30.5555...m/s` to `110km/s`. Yet, when considering the units, no conversion actually has to be made. Simply dividing `220` by `2` would suffice.

Even for the case where the result is desired in another unit, the implementation loses on performance and precision:

```
const auto v = avg_speed(bu::quantity<bu::si::length>(220 * bu::si::kilo * bu::si::meters),
                         bu::quantity<bu::si::time>(2 * hours));
const bu::quantity<miles_per_hour> mph(v);
std::cout << mph.value() << " mi/h\n";
```

Still three conversions are performed, whereas an optimal implementation would store the result of `220km/2h` as `110km/h` without conversion and only convert `110km/h` to `68.35mi/h`.

§ 7.1. Template arguments type deduction

Above problem can be solved using function template argument deduction:

```
template<typename LengthSystem, typename Rep1, typename TimeSystem, typename Rep2>
constexpr auto avg_speed(bu::quantity<bu::unit<bu::length_dimension, LengthSystem>, Rep1> d
                         bu::quantity<bu::unit<bu::time_dimension, TimeSystem>, Rep2> t)
{
  return d / t;
}
```

This allows us to put requirements on the parameter dimensions without limiting the units allowed. Therefore no conversion before the function call is necessary, reducing conversion overhead and precision loss.

Yet, constraining the return value is a bigger problem. In C++17 it is possible to achieve a constrained return value, but the syntax is not very pretty:

```
template<typename LengthSystem, typename Rep1, typename TimeSystem, typename Rep2>
constexpr bu::quantity<typename bu::divide_typeof_helper<
                                        bu::unit<bu::length_dimension, LengthSystem>,
                                        bu::unit<bu::time_dimension, TimeSystem>>::type>
avg_speed(bu::quantity<bu::unit<bu::length_dimension, LengthSystem>, Rep1> d,
          bu::quantity<bu::unit<bu::time_dimension, TimeSystem>, Rep2> t)
{
  return d / t;
}
```

What is more, the user has to manually reimplement dimensional analysis logic in template metaprogramming land, not actually using the units library which should provide such a functionality.

It is worth noting, that for some libraries we cannot even address the first step for the function template arguments. In the case of [NIC_UNITS] derived units are implemented in terms of base units:

```
using meter_t     = units::unit_t<units::unit<std::ratio<1>, units::category::length_unit>>
using kilometer_t = units::unit_t<units::unit<std::ratio<1000, 1>, meter_t>,
                                  std::ratio<0, 1>,
                                  std::ratio<0, 1>>>;
```

This makes it impossible to know upfront where `units::category::length_unit` will exist in a class template instantiation.

§ 7.2. Generic programming with concepts

The answer to constraining templates is again C++20 concepts. With their help the above function can be implemented as:

```
constexpr units::Velocity auto avg_speed(units::Length auto d, units::Time auto t)
{
  return d / t;
}
```

This gives us the benefit of:

- working on the user-provided units and values without any intermediate conversions,
- better error logs (as described in § 6.4 Better errors with C++20 concepts),
- function template parameter constraints clearly expressed in the function template signature,
- possibility to constrain not only the function arguments but also its return type without the need to reimplement the body of the function in a template metaprogramming dialect.

With such an approach, the resulting binary generated by the compiler is the same fast (or sometimes even faster) than the one generated for direct usage of fundamental types.

Additionally, concept usage relieves us from the need to implement a system of quantities, which in other libraries needs to be defined to fix a custom base unit to a specific dimension. In these libraries, defining such a unit system is a workaround for constraining template function parameters and limiting the number of intermediate conversions.

Futhermore it needs to be emphasized, that C++20 concepts are useful not only to constrain function template arguments and their return value but can also be used to constrain the types of user variables:

```
const units::Velocity auto speed = avg_speed(220.km, 2.h);
```

If for some reason the function `avg_speed` would no longer return a velocity, the error would be shown clearly by the compiler, a feature which cannot be provided by C++17 template metaprogramming.

§ 8. `std::ratio` on steroids

Some of the derived units have really big or small ratios. The difference from the base units is so huge that it cannot be expressed with `std::ratio`, which is implemented in terms of `std::intmax_t`.

This makes it really hard to express units like electronvolt (eV) where 1eV = $1.602176634\times10^{-19}$ J or Dalton where 1 Da = $1.660539040(20)\times10^{-27}$ kg. Although a custom system of quantities could be a solution, it would only be a workaround as it cannot provide seamless conversion between all possible units.

A better, more flexible solution is needed. One of the possiblities might be to redefine ratio with one additional parameter:

```
template<std::intmax_t Num, std::intmax_t Den = 1, std::intmax_t Exp = 0>
    requires (Den != 0)
struct new_ratio;
```

With such an approach it will be possible to easily address any occuring ratio with a required precision. For example, the conversion rate between one electronvolt and one Joule could be expressed as:

```
new_ratio<1602176634, 1000000000, -19>
```

§ 9. Extensibility

The units library should be designed in a way that allows users to easily extend it with their own units, derived, or even base dimensions. The C++ Standard Library will most likely decide to ship with a support for "just" physical units with possible extension to digital information dimensions and their units. This should not limit users to the units and quantities provided by library engine, but address all their needs in their specific domains.

The most important points that have to be provided by such C++ Standard library engine in order to provide good extensibility are:

- The library has to be extendible with new units, derived, and base quantities, interacting with the existing ones.
- The user-defined entities have to provide the same user experience as built-in ones.
- Extension shall be possible without preprocessor macros in the user interface.
- Extensions to the C++ Standard library engine created by two independent vendors shall not collide with each other as long as they address separate domains.

§ 10. Easy to use and hard to abuse

Users complain about the complexity of existing solutions. For example, Boost.Units users have to:

- include a lot of specific header files,
- define a lot of types by themselves (see § 7 Limiting intermediate quantity value conversions),
- fight with compilation errors (see § 6 Improving user experience) and debugging,
- define custom systems to workaround intermediate conversions issues (see § 7.2 Generic programming with concepts and § 8 std::ratio on steroids)
- learn lots of library specific behaviors and their side effects (i.e. lack of implicit conversions between units even when it is provable in compile time that such a translation is non-truncating like km -> m),

Most of those issues can be solved during the design time. We should strive to provide:

1. Behavior similar to `std::chrono` as it proved to be a good design and the user base already got used to that.
2. Clear responsibility of each type (base_dimension -> exp -> dimension -> unit -> quantity).
3. Ease to extend with custom dimensions or units.
4. Ease to understand error messages and a good debugging experience thanks to downcast facility and concepts.
5. No dedicated abstraction for systems that would complicate implementation and reasoning about the library engine and functionality (at least until future users will not provide solid requirements and use cases for such an entity).
6. A basic set of prefixes, units, quantities, constants, and concepts.

§ 11. Design principles

The basic design principles that should be used to implement a physical units library for C++ are:

1. Safety and performance:
    - strong types
    - only safe implicit conversions should be allowed
    - compile-time safety and verification wherever possible (break at compile time, not at runtime)
    - constexpr all the things
    - as fast or even faster than working with fundamental types
2. The best possible user experience:
    - interfaces embraced with clear concepts and contracts
    - user friendly compiler errors
    - good debugging experience
3. No macros in the user interface.
4. Easy extensibility.
5. No external dependencies.
6. Possibility to be standardized as a freestanding part of the C++ Standard Library.
7. Batteries included:

- provide basic prefixes, units, quantities, constants, and concepts
- non-experts should easily be able to achieve simple calculations

12. Open questions

12.1. How to represent SI prefixes and derived units?

There are at least 3 ways to represent derived units:

1. Provide a new strong type and an UDL for each unit
2. Define the type only for a coherent derived unit and use multiplier syntax to obtain more derived units
3. Mixed approach using strong types, NTTPs, and variable templates

Starting with the first case. Each derived unit gets its own type and an UDL. With such an approach we can easily write:

```
using units::literals;

const auto d1 = 123km;
const auto d2 = units::quantity<units::kilometre>(123);

const auto v1 = 123kmph;
const auto v2 = units::quantity<units::kilometre_per_hour>(123);
```

The good parts here are:

- clear, strong types for each defined unit
- support for such a unit by the downcasting facility
- existence of an UDL

The drawbacks of such a solution are:

- as library framework will probably not predefine all possible variations of base units and their prefixes (i.e gigametre_per_second) the user will have to define every such a unit before the first use
- the same as above applies to UDLs
- only one unit spelling form (`meter` vs. `meters` vs. `metre` vs. `metres`) supported
- naming of some units can become clunky (i.e. `sq_volt_per_hertz`)

The second case assumes that each dimension will get only a predefined coherent derived unit and the rest of the derived units will be either created with a multiplier syntax or defined by the user:

```
namespace bu = boost::units;

const auto d1 = 123k * bu::si::meters;  // no an actual Boost.Units syntax
const auto d2 = 123 * bu::si::kilo * bu::si::meters;

using kilometer_base_unit = bu::make_scaled_unit<bu::si::length,
                                                 bu::scale<10, bu::static_rational<3>>>::ty
using length_kilometer = kilometer_base_unit::unit_type;
using time_hour = bu::metric::hour_base_unit::unit_type;
using velocity_kilometers_per_hour = bu::divide_typeof_helper<length_kilometer, time_hour>:
BOOST_UNITS_STATIC_CONSTANT(kilometers_per_hour, velocity_kilometers_per_hour);

// const auto v1 = ???
const auto v2 = 123 * kilometers_per_hour;
```

The good parts here are:

- multiple possible spellings of unit names
- easy to use existing coherent derived units with all SI prefixes

The drawbacks of such a solution are:

- verbose UDLs
- no support by the downcasting facility
- user has to make a significant effort to create new derived units that are not easily constructible with SI prefixes (i.e. kilometre)

The third approach is using a mix of several language features including strong types, Non Type Template Parameters (NTTP), and UDLs. With such an approach we can end up with a variety of syntaxes. Please note that the long list below is only to list all of the possibilities in this design space and we do not propose anything like this for now. We can easily forbid any or all of the following syntaxes:

```
inline constexpr auto kilometre = kilo*metre;
inline constexpr auto km = kilometre;

namespace literals {
  constexpr auto operator ""km(unsigned long long l) { return quantity<km, std::int64_t>(l)
  constexpr auto operator ""km(long double l) { return quantity<km, long double>(l); }
}

const auto d1  = quantity<kilo*metre>(123);
const auto d2  = quantity<kilometre>(123);
const auto d3  = quantity<k*metre>(123);
const auto d4  = quantity<k*m>(123);
const auto d5  = quantity<km>(123);
const auto d6  = kilo(123)*metre;
const auto d7  = kilometre(123);
const auto d8  = kilo*metre(123);
const auto d9  = kilometre(123);
const auto d10 = 1000*metre(123);
const auto d11 = metre(123'000);
const auto d12 = 123k*m;
const auto d13 = 123km;
const auto d14 = k*123m;
const auto d15 = 123kilo*metres;
const auto d16 = (km)(123);

const auto v1  = quantity<kilo*metre/hour>(123);
const auto v2  = quantity<kilometre/hour>(123);
const auto v3  = quantity<k*m/h>(123);
const auto v4  = quantity<km/h>(123);
const auto v5  = kilo * metre(123) / hour(1);
const auto v6  = kilo * metre(123) / hour();
const auto v7  = kilometre(123) / hour;
const auto v8  = k*m(123)/h;
const auto v9  = km(123)/h;
const auto v10 = (km/h)(123);
const auto v11 = 123km/h;
```

All of the above variables for length and velocity are respectively of the same type and contain the same value.

The good parts here are:

- library has to implement only base named units
- natural syntax of spelling units (i.e. 123km/h or quantity (123))
- user is able to easily construct any possible variation of base units without any additional library support
- multiple possible spellings of unit names

The drawbacks of such a solution are:

- "there is more than one way to do it" problem
- the code written by the developer will not be similar to the types printed in the compilation error logs

- no support for such a unit by the downcasting facility (units are values instead of types)

§ 12.2. NTTP usage

There are a few points in the physical units domain design that could benefit from Non-Type Template Parameters usage. One of the most apparent cases here is `ratio`. A classical implementation of such a class template looks like this:

```
template<intmax_t Num, intmax_t Den = 1>
struct ratio {
  static constexpr intmax_t num = Num * static_sign<Den>::value / static_gcd<Num, Den>::val
  static constexpr intmax_t den = static_abs<Den>::value / static_gcd<Num, Den>::value;
  using type = ratio<num, den>;
};
```

Besides, it provides a few utilities to do operations on such types:

```
namespace detail {
  template<typename R1, typename R2>
  struct ratio_multiply_impl {
  private:
    static constexpr intmax_t gcd1 = static_gcd<R1::num, R2::den>::value;
    static constexpr intmax_t gcd2 = static_gcd<R2::num, R1::den>::value;
  public:
    using type = ratio<safe_multiply<(R1::num / gcd1), (R2::num / gcd2)>::value,
                       safe_multiply<(R1::den / gcd2), (R2::den / gcd1)>::value>;
    static constexpr intmax_t num = type::num;
    static constexpr intmax_t den = type::den;
  };
}
template<typename R1, typename R2>
using ratio_multiply = detail::ratio_multiply_impl<R1, R2>::type;
```

Usage examples of such an approach looks as follows:

```
struct yard : derived_unit<yard, length, ratio<9'144, 10'000>> {};
struct foot : derived_unit<foot, length, ratio_multiply<ratio<1, 3>, yard::ratio>> {};
struct inch : derived_unit<inch, length, ratio_multiply<ratio<1, 12>, foot::ratio>> {};
struct mile : derived_unit<mile, length, ratio_multiply<ratio<1'760>, yard::ratio>> {};
```

With NTTP the implementation and usage of the `ratio` are much easier:

```
struct ratio {
  std::intmax_t num;
  std::intmax_t den;

  explicit constexpr ratio(std::intmax_t n, std::intmax_t d = 1) :
    num(n * (d < 0 ? -1 : 1) / std::gcd(n, d)),
    den(abs(d) / std::gcd(n, d))
  {
  }

  [[nodiscard]] constexpr bool operator==(const ratio&) = default;

  [[nodiscard]] friend constexpr ratio operator*(const ratio& lhs, const ratio& rhs)
  {
    const std::intmax_t gcd1 = std::gcd(lhs.num, rhs.den);
    const std::intmax_t gcd2 = std::gcd(rhs.num, lhs.den);
    return ratio(safe_multiply(lhs.num / gcd1, rhs.num / gcd2),
                 safe_multiply(lhs.den / gcd2, rhs.den / gcd1));
  }

  [[nodiscard]] friend consteval ratio operator*(std::intmax_t n, const ratio& rhs)
  {
    return ratio(n) * rhs;
  }

  [[nodiscard]] friend consteval ratio operator*(const ratio& lhs, std::intmax_t n)
  {
    return lhs * ratio(n);
  }
};
```

```
// US customary units
struct yard : derived_unit<yard, length, ratio(9'144, 10'000)> {};
struct foot : derived_unit<foot, length, yard::ratio / 3> {};
struct inch : derived_unit<inch, length, foot::ratio / 12> {};
struct mile : derived_unit<mile, length, 1'760 * yard::ratio> {};
```

Also, please see the mixed approach described in § 12.1 How to represent SI prefixes and derived units? which opens the door to new natural syntax of spelling units.

§ 12.3. Relative vs absolute quantity

One of the most critical aspects of the physical units library is to understand what a quantity is? An absolute or relative value? For most dimensions only relative values have sense. For example:

- Where are absolute 123 meters?
- If I am sitting in a moving train, is my velocity == 0?
- Is my velocity == 0 when the train stops?

However, for some base quantities like temperature, absolute values are really needed. For example, how much is `0 °C + 0 °C`? Is it `0 °C` or `0 °C` or `273.15 °C`? Yes, the repeated value of `0 °C` is not an error here ;-) Actually, all of the answers are right:

- Two (absolute) temperatures:

```
0 °C + 0 °C = 273.15 K + 273.15 K = 546.30 K = 273.15 °C
```

- An (absolute) temperature and a (relative) temperature interval:

```
0 °C + 0 °C = 273.15 K + 0 K = 273.15 K = 0 °C
```

- Two (relative) temperature intervals:

```
0 °C + 0 °C = 0 K + 0 K = 0 K = 0 °C
```

As proven above, it is a complex and a pretty hard problem. The average user of the library will probably not be able to distinguish between different kinds of quantities. This is why it was decided that only relative quantity values will be modeled by the library. Moreover, providing support for only relative quantities of other temperature units than Kelvin will probably still be misused by the users. This is why we suggest to support only Kelvins as built-in temperature units and provide verbose non-member utility functions for conversions between different kinds of temperature values and their units.

§ 12.4. Should we support systems as a separate type?

US Customary System has the same dimensions and most of the units as the SI with the differences scoped mostly only in length and mass units and derived quantities using those. From the implementation and standardization point of view it is much simpler to use the common definitions of such physical dimensions and just provide units dedicated to such a system next to the SI ones (i.e. meters and miles).

Even systems that seem to be totally isolated from typical SI uses cases like `coffee/milk/water/sugar` system at some point will probably need time, volume, and other SI dimensions too.

Boost.Units uses systems mostly to provide the capability of having a different base unit for a dimension to limit intermediate conversions while passing quantities as vocabulary types in the interfaces. Usage of templates functions constrained with concepts for generic algorithms and concrete types for domain-specific needs addresses this area easily. For more information on this subject please refer to § 7 Limiting intermediate quantity value conversions.

Important point to note here is that adding direct systems support in the library type system might negatively affect user experience. Most of the verbose compilation errors presented in § 6.1 Type aliasing issues are caused by a dedicated systems support in Boost.Units.

§ 12.5. Interoperability with `std::chrono::duration`

One of the most challenging problems to solve in the physical units library will be the interoperability with `std::chrono::duration`. `std::chrono` is an excellent library and has a wide adoption in the industry right now. However it has also some issues that make its design not suitable for a general purpose units library framework:

1. It addresses only one of many dimensions, namely time. There is no possibility to extend it with other dimensions support.
2. `quantity` class template needs a few more member functions to provide support for conversions between different dimensions.
3. SG6 members raised an issue with `std::chrono::duration` returning `std::common_type_t<Rep1, Rep2>` from most of the arithmetic operators. This does note play well with custom representation types that return different type in case of multiplication and different in case of division operation.
4. `std::ratio` is not able to handle large prefixes required by some units (more information in § 8 std::ratio on steroids).

Because of the above issues we cannot just use `std::chrono::duration` design as it is right now and use it for physical units implementation or even as a representation of only time dimension. There are however, a few possibilities here to provide interoperability between the types:

1. One of the solutions could be making a `std::chrono::duration` an alias or a child class of `std::units::quantity` class template (assuming that we will not use NTTP ratios as described in § 12.2 NTTP usage). This would be probably the best solution from the API point of view but unfortunately it will cause an ABI break.
2. Provide a partial specialization for a time dimension to have additional conversion operations to/from `std::chrono::duration` built-in into the class template itself for a time dimension. However, this solution would be hard to maintain and keep synchronized with the `std::units::quantity` class template.
3. Provide non-member function to convert and compare between those two types.
4. Just ignore `std::chrono::duration` and do not provide any conversion utilities in the standard library.

From all options above we propose the first one if we decide that C++23 will be an ABI breaking release. In such a case we could update `std::ratio` type as described in § 8 std::ratio on steroids. Otherwise, we would probably go with the option #3.

§ 12.6. Should we provide integral UDLs?

User Defined Literals support is really handy for the end-users. However, it sometimes might cause more confusion than benefits. For example defining both UDL versions for a velocity:

```
inline namespace literals {

  constexpr auto operator""kmph(unsigned long long l)
  { return quantity<kilometre_per_hour, std::int64_t>(l); }

  constexpr auto operator""kmph(long double l)
  { return quantity<kilometre_per_hour, long double>(l); }

}
```

and a function template defined as:

```
constexpr units::Velocity auto avg_speed(units::Length auto d, units::Time auto t)
{
  return d / t;
}
```

might cause the following code to not compile:

```
const units::Velocity auto speed = avg_speed(220km, 2h);
```

while the following one compiles fine:

```
const units::Velocity auto speed = avg_speed(220.km, 2h);
```

Above is caused by the constraints copied from `std::chrono::duration` and put on the conversion constructors requiring the denominator of `ratio` to be 1 in case of the integral representation type.

Based on the above we could agree to provide an UDL for floating point literals only or make the integral one return a quantity with `long double` as a representation type. However, if we consider a base quantity like a digital information, what does it mean to have a fraction of bit? This probably not the only one isolated example when actually only integral UDLs have sense.

Summarizing above we have the following options to choose from as an answer to "Should we provide integral UDLs?":

1. Yes, as is (always both integral and floating-point for all units). And leave it up to the user to use them correctly.
2. Yes, but integral literals get floating-point `Rep`.
3. Yes, but only for specific units like a `bit`, `byte`, etc. where floating-point types do not have much sense (no floating-point UDLs in such case).
4. No, just use floating-point UDLs for all (no integral UDLs at all in the library).

§ 12.7. `quantity<dim_length, metre>` or `quantity<metre>`?

The initial version of the mp-units library provided the following `quantity` class template definition:

```
template<Dimension D, Unit U, Scalar Rep>
  requires std::same_as<D, typename U::dimension>
class quantity;
```

This allowed the following helper aliases:

```
template<Unit U = meter, Scalar Rep = double>
using length = quantity<dimension_length, U, Rep>;
```

With such a framework and CTAD usage user could write the following:

```
units::length d(3);                // 3 meters
units::length<units::mile> d3(3);  // 3 miles
```

or

```
units::velocity speed = avg_speed(220.km, 2.h);
```

or

```
template<typename U, typename Rep>
void foo(units::length<U, Rep> dist);
```

to constrain the type to a length dimension.

The downside of such a design was that the dimension was provided twice in every `quantity` class template instantiation which was affecting user experience by longer types in error logs or during debugging:

```
error: conversion from ‘quantity<units::dimension<units::exp<units::base_dim_length, 1>,
units::exp<units::base_dim_time, 1> >, units::unit<units::dimension<units::exp<
units::base_dim_length, 1>, units::exp<units::base_dim_time, 1> >, std::ratio<3600000, 1> >
[...]>’ to non-scalar type ‘quantity<units::dimension_velocity, units::kilometer_per_hour,
[...]>’ requested
```

During evening session in Cologne the author received a feedback from SG6 members that such a duplication should be removed. Right now the design looks as follows:

```
template<Unit U, Scalar Rep>
class quantity;
```

With this there is no possibility to provide a helper alias for a length dimension and above examples have to be implemented in terms of concepts:

```
units::quantity<units::metre> d(3);  // 3 meters
units::quantity<units::mile> d3(3);  // 3 miles
```

or

```
units::Velocity auto speed = avg_speed(220.km, 2.h);
```

or

```
template<units::Length Quantity>
void foo(Quantity dist);
```

The good part here is that the error logs are more readable with such an approach:

```
error: conversion from ‘quantity<units::unit<units::dimension<units::exp<units::base_dim_le
1>, units::exp<units::base_dim_time, 1> >, std::ratio<3600000, 1> >, [...]>’ to non-scalar
‘quantity<units::kilometer_per_hour, [...]>’ requested
```

Both cases provide the similar functionality so it is a matter of taste here on which of the syntaxes the Committee will choose to continue with.

§ 12.8. Should we provide `seconds<int>` or stay with `quantity<second, int>`?

Some of the users complain that writing `quantity<second>(123)` is too verbose and they would prefer a helper alias that would allow them to write `seconds(123)`. This however, starts to generate a few issues:

- `second` represent a unit
- `seconds` represents a quantity
- plural form reserved for quantities might not be easy to achieve or easy to distinguish for some units like `sq_volt_per_hertz`.

We can consider renaming `second` to `unit_second` and provide `seconds` as an alias to the `quantity` class template. However, this will probably set in stone usage of aliases as no one will be willing to write a verbose code like `quantity<unit_second>(123)`. This is why we are looking for a concrete guideline on which of the options the Committee prefers.

Author preference is to stay with the current design and leave it up to the users to create any helper aliases for their domains and use cases if they choose so (i.e. `s(123)`).

§ 12.9. Should we provide support for dimensionless quantities?

Some quantities of dimension one are defined as the ratios of two quantities of the same kind. The coherent derived unit is the number one, symbol 1. For example: plane angle, solid angle, refractive index, relative permeability, mass fraction, friction factor, Mach number, etc.

Numbers of entities are also quantities of dimension one. For example: number of turns in a coil, number of molecules in a given sample, degeneracy of the energy levels of a quantum system.

Should the library treat such entities as regular scalars or should some strong typing mechanism be provided to support those?

§ 13. Impact on the Standard

The library would be mostly a pure addition to the C++ Standard Library with the following potential exceptions:

1. It is unclear how to provide interoperability with the `std::chrono::duration` (more information in § 12.5 Interoperability with std::chrono::duration).
2. `std::units::ratio` will most probably need to be a different type with the similar semantics to `std::ratio` (more information in § 8 std::ratio on steroids). However, if we decide C++23 to be an ABI breaking release we could update `std::ratio` with an additional template parameter.

§ 14. Implementation Experience

The author of this document implemented `mp-units` [MP-UNITS] library, where he tested different ideas and proved the implementability of the features described in the paper. The library framework consists of a few concepts: quantities, units, dimensions, and their exponents. From the user's point of view, the most important is a quantity.

Quantity is a precise amount of a unit for a specified dimension with a specific representation:

```
units::quantity<units::kilometre, double> d1(123);
auto d2 = 123km;    // units::quantity<units::kilometre, std::int64_t>
```

There are C++ concepts provided for each such quantity type:

```
template<typename T>
concept Length = QuantityOf<T, length>;
```

With these concepts, we can easily write a function template:

```
constexpr units::Velocity auto avg_speed(units::Length auto d, units::Time auto t)
{
  return d / t;
}
```

This template function can be used in the following way:

```
const units::quantity<units::kilometre> d(220);
const units::quantity<units::hour> t(2);
const units::Velocity auto kmph = units::quantity_cast<units::kilometre_per_hour>(avg_speed
std::cout << kmph.count() << " km/h\n";

const units::Velocity auto speed = avg_speed(140.mi, 2.h);
assert(speed.count() == 70);
std::cout << units::quantity_cast<units::mile_per_hour>(speed).count() << " mph\n";
```

This guarantees that no intermediate conversions are being made, and the output binary is as effective as implementing the function with `double`s.

Additionally, thanks to the extensive usage of the C++ concepts and the downcasting facility, the library provides an excellent user experience. The error message for type aliases would look like:

```
[with D = units::quantity<units::unit<units::dimension<units::exp<units::base_dim_length, 1
               units::exp<units::base_dim_time, 1, -1> >, units::ratio<5, 18> >, double>]
```

Yet, thanks to downcast facility, the actual error message is:

```
[with D = units::quantity<units::kilometre_per_hour, double>]
```

The breakpoint in the debugger became readable as well:

```
Breakpoint 1, avg_speed<units::quantity<units::kilometre, double>,
                        units::quantity<units::hour, double> >
(d=..., t=...) at velocity.cpp:31
31      return d / t;
```

Moreover, it is really easy to extend the library with custom units, derived units, and base dimensions. For example, if the user wants to provide a custom `digital information` base dimension and new units based on it, only minimal code is required:

```
#include <units/quantity.h>

using namespace units;

// custom base dimension
struct base_dim_digital_information {
  static constexpr const char* value = "digital information";
};

// custom derived dimension and its concept
struct digital_information : derived_dimension<digital_information,
                                               units::exp<base_dim_digital_information, 1>>

template<typename T>
concept DigitalInformation = QuantityOf<T, digital_information>;

// custom units and their units
struct bit : derived_unit<bit, digital_information> {};
struct byte : derived_unit<byte, digital_information, ratio<8>> {};

inline namespace literals {
  constexpr auto operator""_b(unsigned long long l) { return quantity<bit, std::int64_t>(l)
  constexpr auto operator""_B(unsigned long long l) { return quantity<byte, std::int64_t>(l
}

// unit tests
static_assert(1_B == 8_b);
```

§ 15. Polls

1. Do we want a physical units library in the C++ standard?
2. Should we provide the support for some off-system units (i.e. eV)?
3. Do we want to have support for digital information dimensions and its prefixes in the initial version of the library?
4. Do we prefer UDL, multiply, or mixed syntax for units?
5. Do we like the concept-based approach to prevent truncation?

6. Do we like a downcasting facility or do we want to wait for other solutions (strong types in the language, better compiler errors, ...)?
7. Do we prefer NTTP usage for `ratio` and `exp`?
8. Do we want to require explicit representation casts between different units of the same dimension, or do we allow `chrono`-like implicit conversions (floating-point representation and non-truncating integer conversions)?
9. Do we want to require explicit unit casts between different units of the same dimension, or do we allow `chrono`-like implicit conversions (implicitly convert kilometre to metre, sometimes doing unnecessary conversions)?
10. Do we agree with Kelvins only support for temperature and verbose conversion functions for other units and absolute temperatures?
11. Which option of UDLs do we prefer (§ 12.6 Should we provide integral UDLs?)?
12. Should American spelling be provided? (`meter` vs. `metre`, `ton` vs. `tonne`, ...)
13. Should we provide `seconds<int>` or stay with `quantity<second, int>` (§ 12.8 Should we provide seconds<int> or stay with quantity<second, int>?)?
14. Should we provide support for dimensionless quantities (§ 12.9 Should we provide support for dimensionless quantities?)?
15. Do we want to introduce a dedicated system type?
16. Should affine types be provided? (relative vs absolute)
17. Should ISO 80000-1:2009 units be provided? If yes, how should updates to the ISO standard be handled? (Separate namespaces?)
18. Should constants be provided? If yes, how should updates to the constants be handled? (Separate namespaces?)

§ 16. Acknowledgments

Special thanks and recognition goes to Epam Systems for supporting my membership in the ISO C++ Committee and the production of this proposal.

I would also like to thank Jan A. Sende for his contributions to the `mp-units` library and this document.

§ Index

§ Terms defined by this specification

system, in §3.1

system of quantities, in §3.1

system of units, in §3.1

unit, in §3.1

unit of measurement, in §3.1

value, in §3.1

value of a quantity, in §3.1

§ References

§ Normative References

[ISO_80000-1]
Quantities and units - Part 1: General. URL: https://www.iso.org/standard/30669.html

§ Informative References

[BENRI]
Jan A. Sende. benri. URL: https://github.com/jansende/benri

[BOOST.UNITS]
Steven Watanabe; Matthias C. Schabel. Boost.Units. URL: https://www.boost.org/doc/libs/1_70_0/doc/html/boost_units.html

[BRYAN_UNITS]
Bryan St. Amour. units. URL: https://github.com/bstamour/units

[CLARENCE]
Steve Chawkins. Mismeasure for Measure. URL: https://www.latimes.com/archives/la-xpm-2001-feb-09-me-23253-story.html

[COLUMBUS]
Christopher Columbus. URL: https://en.wikipedia.org/wiki/Christopher_Columbus

[CPPNOW17-UNITS]
Steven Watanabe. cppnow17-units. URL: https://github.com/swatanabe/cppnow17-units

[DISNEY]
Cause of the Space Mountain Incident Determined at Tokyo Disneyland Park. URL: https://web.archive.org/web/20040209033827/http://www.olc.co.jp/news/20040121_01en.html

[FLIGHT_6316]
Korean Air Flight 6316 MD-11, Shanghai, China - April 15, 1999. URL: https://ntsb.gov/news/press-releases/Pages/Korean_Air_Flight_6316_MD-11_Shanghai_China_-_April_15_1999.aspx

[GIMLI_GLIDER]
Gimli Glider. URL: https://en.wikipedia.org/wiki/Gimli_Glider

[MARS_ORBITER]
Mars Climate Orbiter. URL: https://en.wikipedia.org/wiki/Mars_Climate_Orbiter

[MIKEFORD3_UNITS]
Michael Ford. units. URL: https://github.com/mikeford3/units

[MP-UNITS]
Mateusz Pusz. mp-units. URL: https://github.com/mpusz/units

[NIC_UNITS]
Nic Holthaus. units. URL: https://github.com/nholthaus/units

[PHYSUNITS-CT-CPP11]
Martin Moene. PhysUnits-CT-Cpp11. URL: https://github.com/martinmoene/PhysUnits-CT-Cpp11

[WILD_RICE]

Manufacturers, exporters think metric. URL: https://www.bizjournals.com/eastbay/stories/2001/07/09/focus3.html